PROJECT REFERENCE DOCUMENT

Ambrosia

Restaurant Order & Kitchen Management System

ASP.NET Core 8 | PostgreSQL 16 | Angular 17 | SignalR

Backend	Database	Frontend	Real-time
ASP.NET Core 8	PostgreSQL 16	Angular 17	SignalR

Version 1.0 | Internship Capstone Project | 2024

Dine-in QR Ordering • Kitchen Queue Management • Billing • Inventory

1. Project Overview

Ambrosia is a dine-in restaurant management system. Customers scan a QR code at their table which creates a guest session, allowing them to self-order without any account or login. The system manages the kitchen queue in real time via SignalR, handles billing, and tracks inventory. The architecture is delivery-agnostic — the schema is future-ready for delivery as a Phase 2 addition without breaking existing structure.

Layer	Technology	Responsibility
Presentation	Angular 17 + NgRx	UI, routing, state management, SignalR client
API	ASP.NET Core 8 Web API	Endpoints, JWT auth, FluentValidation, Swagger
Service	C# Services + AutoMapper	Business logic, domain rules, DTO mapping
Repository	EF Core 8 + LINQ	Data access, migrations, query filters
Database	PostgreSQL 16	Persistence, triggers, functions, views
Real-time	SignalR	Live kitchen/order/table status updates

1.1 Primary Flow (Current Scope)

```
Customer scans QR code at table
-> Guest session JWT issued (3 hours / until order completed)
-> Customer browses menu and places order
-> Kitchen receives order via SignalR
-> Kitchen marks InPrep -> Ready
-> Admin generates bill -> marks Paid
-> Order Completed -> Table back to Available -> Guest session ends
```

1.2 Future Scope (Phase 2 — Not Built Now)

Delivery ordering, customer account-based ordering, and guest session blacklisting on order completion are Phase 2 concerns. The schema accommodates these via nullable CustomerId and TableId on Orders, and the SessionType field. No Phase 2 work is in scope now.

2. Database Schema

2.1 Roles

Column	Type	Constraints	Notes
Id	INT	PK, Identity	Seeded: 1=Admin, 2=Chef
Name	VARCHAR(50)	NOT NULL, UNIQUE, stored as string	Enum: Admin, Chef — Customer is Phase 2
CreatedAt	TIMESTAMPTZ	NOT NULL, fixed seed date	Never use UtcNow in seed data

2.2 Users

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
Name	VARCHAR(100)	NOT NULL	Trimmed before save
Email	VARCHAR(200)	NOT NULL, UNIQUE	Normalized to lowercase
PasswordHash	VARCHAR(500)	NOT NULL	BCrypt workFactor 12
RoleId	INT	FK Roles — Restrict	
Phone	VARCHAR(15)	NULLABLE	Mandatory for Customer role (Phase 2)
Address	VARCHAR(300)	NULLABLE	Mandatory for Customer role (Phase 2)
IsDeleted	BOOLEAN	DEFAULT false	Soft delete — query filter applied
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	Auto-set by SaveChangesAsync override

2.3 Tables

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
Number	INT	NOT NULL, UNIQUE	Physical table number on floor
Status	VARCHAR(20)	DEFAULT Available	Enum: Available, Occupied, Reserved
Capacity	INT	NOT NULL	
IsActive	BOOLEAN	DEFAULT true	Inactive = not shown on floor view
QRCodeUrl	VARCHAR(500)	NULLABLE	Generated on table creation — Azure Blob URL
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	

2.4 Categories

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
Name	VARCHAR(100)	NOT NULL, UNIQUE	
IsActive	BOOLEAN	DEFAULT true	Temp hide from menu without deleting
IsDeleted	BOOLEAN	DEFAULT false	Permanent — query filter applied
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	

2.5 MenuItems

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
CategoryId	INT	FK Categories — Restrict	
Name	VARCHAR(150)	NOT NULL	
Description	VARCHAR(500)	NULLABLE	
Price	NUMERIC(10,2)	NOT NULL, CHECK > 0	
ImageUrl	VARCHAR(500)	NULLABLE	Azure Blob Storage URL
PreparationTime	INT	NOT NULL, 1–120 mins	Used for ETA display + KDS sort order
IsAvailable	BOOLEAN	DEFAULT true	Chef/Admin toggle, also auto-set by Inventory
IsDeleted	BOOLEAN	DEFAULT false	Query filter applied
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	

2.6 Inventory

One record per MenuItem. Tracks today's portion count. Admin manually enters quantity. System decrements on each order placed.

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
MenuItemId	INT	FK MenuItems — Cascade, UNIQUE	One row per item
QuantityAvailable	INT	NOT NULL, DEFAULT 0	Portions remaining today
LowStockThreshold	INT	NOT NULL, DEFAULT 10	Alert triggers below this
IsAutoDisabled	BOOLEAN	DEFAULT false	True if stock=0 caused the disable

LastUpdatedBy	INT	FK Users — Restrict	Admin who last updated
UpdatedAt	TIMESTAMPTZ	DEFAULT NOW()	

2.7 Orders

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
CustomerId	INT	FK Users, NULLABLE	NULL for guest QR orders. Used in Phase 2
TableId	INT	FK Tables, NULLABLE	NULL reserved for future delivery orders
SessionType	VARCHAR(20)	NOT NULL	Values: guest / staff (customer = Phase 2)
Status	VARCHAR(20)	DEFAULT Pending	Enum — see transition table
TotalAmount	NUMERIC(10,2)	DEFAULT 0	Server-computed from OrderItems — never from client
Notes	VARCHAR(300)	NULLABLE	Order-level notes (birthday, allergies)
IsQROrder	BOOLEAN	DEFAULT false	Distinguishes QR vs manual staff entry
CompletedAt	TIMESTAMPTZ	NULLABLE	Set when Completed or Cancelled
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	

2.8 OrderItems

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
OrderId	INT	FK Orders — Cascade	INDEX on this — most frequent join
MenuItemId	INT	FK MenuItems — Restrict	
Quantity	INT	NOT NULL, 1–10	
UnitPrice	NUMERIC(10,2)	NOT NULL	Snapshotted from MenuItem.Price at order time
Notes	VARCHAR(300)	NULLABLE	Item-level notes: no onions, extra spicy
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	

2.9 Bills

Column	Type	Constraints	Notes
Id	INT	PK, Identity	
OrderId	INT	FK Orders — Cascade, UNIQUE	One bill per order — DB enforced

SubTotal	NUMERIC(10,2)	NOT NULL	Sum of Qty * UnitPrice
TaxRate	NUMERIC(5,4)	NOT NULL, DEFAULT 0.18	0.1800 = 18% GST
DiscountAmt	NUMERIC(10,2)	DEFAULT 0	Manual discount or coupon
FinalAmt	NUMERIC(10,2)	NOT NULL	SubTotal*(1+TaxRate) - DiscountAmt
Status	VARCHAR(20)	DEFAULT Pending	Enum: Pending, Paid, Failed
CreatedAt	TIMESTAMPTZ	DEFAULT NOW()	
UpdatedAt	TIMESTAMPTZ	NULLABLE	

3. Entity Relationships & Delete Behaviour

Relationship	Type	FK On	OnDelete	Reason
User → Role	N:1	User.RoleId	Restrict	Cannot delete role while users exist
MenuItem → Category	N:1	MenuItem.CategoryId	Restrict	Cannot delete category with live items
Order → User	N:1	Order.CustomerId	Restrict	Cannot delete user with order history
Order → Table	N:1	Order.TableId	Restrict	Cannot delete table with order history
OrderItem → Order	N:1	OrderItem.OrderId	Cascade	Delete order = delete its items
OrderItem → MenuItem	N:1	OrderItem.MenuItemId	Restrict	Item data preserved in order history
Bill → Order	1:1	Bill.OrderId	Cascade	Delete order = delete its bill
Inventory → MenuItem	1:1	Inventory.MenuItemId	Cascade	Delete item = delete stock record
Inventory → User	N:1	Inventory.LastUpdatedBy	Restrict	Audit trail preserved

3.1 Order Status Transitions

Valid transitions enforced in service layer. Any other jump returns HTTP 400.

```
Pending -> InPrep Kitchen clicks Start
Pending -> Cancelled Admin cancels before prep begins
InPrep -> Ready Kitchen clicks Done
InPrep -> Cancelled Admin cancels during prep
Ready -> Completed Bill paid, Admin closes order
Completed -> [TERMINAL] No further transitions allowed
Cancelled -> [TERMINAL] No further transitions allowed
```

3.2 Soft Delete vs Status vs Toggle — Per Entity

Entity	Deletion Type	Who	Visible When Off	Notes
User	IsDeleted (soft)	Admin	Admin audit only	Query filter auto-excludes
Role	No deletion	Nobody	Always	Seeded, fixed reference data
Table	IsActive toggle	Admin	Not on floor view	IsActive not IsDeleted
Category	IsDeleted + IsActive	Admin	Admin panel only	IsActive = temporary hide
MenuItem	IsDeleted + IsAvailable	Admin/Chef	Admin sees all	IsAvailable auto-set by inventory
Order	Status only	Admin	All — filtered by screen	Never physically deleted
OrderItem	No deletion	Nobody	Always with Order	No independent lifecycle
Bill	Status only	Admin	All — for reports	Never physically deleted

Inventory	No deletion	Nobody	Always with MenuItem	No independent lifecycle

4. Authentication & Authorization

4.1 JWT Configuration

Setting	Value	Notes
Access token expiry	15 minutes	Short-lived, refreshed automatically
Refresh token expiry	7 days	Stored in HttpOnly cookie — not localStorage
Guest token expiry	3 hours	Or until order Completed — whichever comes first
Password hashing	BCrypt	workFactor 12
Login lockout	5 attempts	Locks for 15 minutes after 5 failed attempts in 10 min
JWT secret	256-bit min	Stored in Azure Key Vault — never in appsettings.json

4.2 Session Types

```
Guest (QR scan) -> sessionType=guest, tableId=4 (no userId, no role)
Staff (login) -> sessionType=staff, userId=2, role=Admin
userId=3, role=Chef
Customer (Phase 2)-> sessionType=customer, userId=7, role=Customer
```

For guest sessions, tableId is embedded in the JWT claim and extracted server-side. It is never accepted from the request body. A guest cannot claim to order for a different table.

4.3 Guest Session Flow

```
1. Customer scans QR: https://app.ambrosia.com/qr?table=4
2. Angular reads tableId=4 from URL query param
3. POST /api/auth/guest-session { tableId: 4 }
4. Backend: validate table exists and IsActive=true
5. Issue 3-hour JWT with claims: { sessionType: guest, tableId: 4, jti: uuid }
6. Angular stores token in sessionStorage (dies on tab close)
7. HTTP interceptor attaches token to all subsequent requests
8. On order placement: tableId read from JWT claim — not request body
9. Session ends on: JWT expiry (3hr) OR tab close (sessionStorage cleared)
```

4.4 Role Permissions

Role	Can Access
Admin	All endpoints — menu, orders, billing, reports, inventory, user management, tables
Chef	Kitchen queue (read + start/done), item availability toggle only
Guest	Browse menu (public), place order (guest JWT), view own table order status
Customer	Phase 2 — self-register, login, place order, order history

4.5 CanPlaceOrder Policy

A custom ASP.NET Core Authorization Policy called CanPlaceOrder allows requests through if the caller is a valid guest JWT, customer JWT, or admin JWT. The policy handler reads the sessionType claim to decide. The service layer then enforces identity — tableId from JWT claim for guests, userId from JWT claim for customers. Neither is accepted from the body.

5. API Endpoints

Base URL: /api | Non-public endpoints require: Authorization: Bearer <token>

5.1 Auth — /api/auth

Method	Endpoint	Description	Auth
POST	/register	Admin creates staff (Chef) account	Admin
POST	/login	Login — returns JWT + refresh token	Public
POST	/refresh	Renew access token via refresh token	Public
POST	/logout	Invalidate refresh token	Required
GET	/me	Current user profile	Required
POST	/guest-session	Issue guest JWT for QR table scan	Public

5.2 Users — /api/users

Method	Endpoint	Description	Auth
GET	/	All staff users — paginated	Admin
GET	/{id}	Single user	Admin
PUT	/{id}	Update name or role	Admin
PATCH	/{id}/password	Change password	Admin / Self
DELETE	/{id}	Soft delete user (IsDeleted=true)	Admin

5.3 Tables — /api/tables

Method	Endpoint	Description	Auth
GET	/	All tables with current status	Admin
GET	/{id}	Single table	Admin
GET	/{id}/orders	Active orders on this table	Admin / Chef
POST	/	Create table + auto-generate QR code	Admin
PUT	/{id}	Update table number or capacity	Admin
PATCH	/{id}/status	Manually update table status	Admin
PATCH	/{id}/active	Toggle IsActive	Admin

5.4 Categories — /api/categories

Method	Endpoint	Description	Auth
GET	/	All non-deleted categories	Public
GET	/{id}	Single category	Public
POST	/	Create category	Admin
PUT	/{id}	Update category name	Admin
PATCH	/{id}/active	Toggle IsActive	Admin
DELETE	/{id}	Soft delete category	Admin

5.5 Menu Items — /api/menu

Method	Endpoint	Description	Auth
GET	/	Available items — paginated + filterable	Public
GET	/{id}	Single item details	Public
POST	/	Create menu item	Admin
PUT	/{id}	Update item details	Admin
PATCH	/{id}/availability	Toggle IsAvailable	Admin / Chef
DELETE	/{id}	Soft delete item	Admin

Query params on GET /: ?categoryId=&search=&page=&pageSize=&sortBy=price&isAvailable=true

5.6 Orders — /api/orders

Method	Endpoint	Description	Auth
GET	/	All orders — paginated, filter by status/table/date	Admin
GET	/active	All non-completed orders	Admin / Chef
GET	/{id}	Order detail with all items	Admin / Chef
POST	/	Place order (guest, staff)	CanPlaceOrder
PATCH	/{id}/status	Update status — validated transitions only	Admin / Chef
POST	/{id}/items	Add items to existing Pending order	Admin
DELETE	/{id}/items/{itemId}	Remove item from Pending order	Admin
DELETE	/{id}	Cancel order — sets Status=Cancelled	Admin

Query params: ?status=&tableId=&from=&to=&page=&pageSize=

5.7 Kitchen — /api/kitchen

Method	Endpoint	Description	Auth
GET	/queue	Live queue sorted by urgency (fn_get_kitchen_queue)	Admin / Chef
PATCH	/queue/{id}/start	Mark order InPrep	Chef
PATCH	/queue/{id}/complete	Mark order Ready	Chef
GET	/queue/stats	Active count, overdue count, average prep time	Admin

5.8 Bills — /api/bills

Method	Endpoint	Description	Auth
GET	/{id}	Bill details	Admin
GET	/order/{orderId}	Bill for a specific order	Admin
POST	/order/{orderId}	Generate bill (order must be Ready status)	Admin
PATCH	/{id}/pay	Mark bill as Paid	Admin
PATCH	/{id}/discount	Apply discount amount	Admin

5.9 Inventory — /api/inventory

Method	Endpoint	Description	Auth
GET	/	All items with stock levels	Admin / Chef
GET	/{menuItemId}	Stock for specific item	Admin / Chef
PUT	/{menuItemId}	Update stock quantity — manual entry	Admin
GET	/low-stock	Items below LowStockThreshold	Admin / Chef

5.10 Reports — /api/reports

Method	Endpoint	Description	Auth
GET	/revenue/daily	Revenue for a specific date (?date=)	Admin
GET	/revenue/range	Revenue over date range (?from=&to=)	Admin
GET	/orders/summary	Order count, cancellation rate, avg value	Admin
GET	/menu/top-items	Top N selling items by quantity (?limit=)	Admin
GET	/menu/category-performance	Revenue and orders per category	Admin
GET	/kitchen/sla	% orders completed within prep time SLA	Admin
GET	/tables/turnover	Orders per table per day	Admin

6. SignalR — Real-Time Architecture

6.1 Hub Groups

Group	Who Joins	Receives
KitchenGroup	All Chef screens	New orders, overdue alerts, item unavailable
AdminGroup	Admin dashboard screens	Live counters, order ready, overdue alerts
Table-{tableId}	Guest browser tab	Own order status updates only

6.2 Server to Client Events

Event	Fired When	Target	Payload Summary
NewOrderReceived	Order placed	Kitchen + Admin	orderId, tableNo, items, createdAt
OrderStatusChanged	Any status update	Table-{id} + Admin	orderId, newStatus, message
OrderReady	Status changes to Ready	Admin	orderId, tableNo
ItemUnavailable	IsAvailable set to false	KitchenGroup	menuItemId, itemName
OrderOverdue	SLA breach detected	Kitchen + Admin	orderId, tableNo, overdueByMins
BillGenerated	Bill created for order	Table-{id}	billId, finalAmt

6.3 Priority & SLA Logic

SignalR is the delivery channel, not the calculator. Priority is computed elsewhere and pushed via SignalR.

Concern	Where Handled	Detail
Queue sort order	DB function fn_get_kitchen_queue()	Sorted by CreatedAt + MAX(PrepTime) ascending
SLA breach detection	Background worker (IHostedService)	Runs every 60s, fires OrderOverdue if overdue
Client re-sort	Angular setInterval every 30s	Re-orders cards locally without server round-trip
ETA display	DB function fn_get_estimated_wait()	MAX(PrepTime) of items — shown on order confirmation

7. Database Automation

7.1 Triggers

Trigger	Fires On	Action
trg_order_placed	AFTER INSERT ON Orders	Set Tables.Status = Occupied for TableId
trg_order_completed	AFTER UPDATE Status ON Orders	If no active orders remain on table -> Available
trg_generate_bill	AFTER UPDATE Status ON Orders	Status = Ready -> INSERT Bills row (Pending)
trg_lock_closed_orders	BEFORE UPDATE ON Orders	RAISE EXCEPTION if current status is terminal
trg_guard_item_delete	BEFORE UPDATE IsDeleted MenuItems	Block if item exists in any active order

7.2 Stored Functions

Function	Returns	Purpose / Called By
fn_get_estimated_wait(orderId)	INT minutes	MAX(PrepTime) of items — order confirmation ETA
fn_get_kitchen_queue()	TABLE	KDS screen — sorted by urgency, includes IsOverdue flag
fn_daily_revenue(date)	TABLE	Reports — revenue, avg value, cancelled count for date

7.3 Views

View	Purpose
vw_active_orders	Admin/order board — all non-completed orders joined with table, user, item count, max prep time
vw_top_selling_items	Reports — menu items ranked by quantity sold with total revenue, joined to category

8. EF Core — Fluent API Configuration Rules

8.1 Configuration Rules

- All enums stored as strings via .HasConversion<string>() — Orders.Status, Tables.Status, Bills.Status, Roles.Name
- All money columns use .HasColumnType("numeric(10,2)") — Price, UnitPrice, TotalAmount, SubTotal, DiscountAmt, FinalAmt
- TaxRate uses numeric(5,4) — stores 0.1800 for 18% GST
- Soft delete query filters applied on User and MenuItem — auto-excludes IsDeleted=true from all queries
- IgnoreQueryFilters() used in Admin service methods only — to see deleted records for audit
- SaveChangesAsync override auto-sets UpdatedAt on all Modified entities via ChangeTracker loop
- Role seed uses fixed DateTimeOffset — never DateTimeOffset.UtcNow (causes false migration diffs on every add-migration)
- Bills.OrderId has UNIQUE index — enforces one bill per order at DB level, not just application level
- WithMany() always references the collection navigation property — .WithMany(r => r.Users), never .WithMany()
- WithOne() always references the navigation on the other side — .WithOne(o => o.Bill)

8.2 Critical Indexes

Index	Type	Reason
Users.Email	UNIQUE	Fast login lookup, prevents duplicate accounts
Orders.Status	Standard	Filter active/pending orders — most frequent query
Orders.TableId	Standard	Get orders by table — used by QR status screen
Orders.CreatedAt	Standard	Date range queries for reports
OrderItems.OrderId	Standard	Fetch items by order — most frequent join in system
Bills.OrderId	UNIQUE	One bill per order enforced at database level
MenuItems.CategoryId	Standard	Filter menu by category — QR menu browse
Inventory.MenuItemId	UNIQUE	One inventory record per menu item

9. Validation & Security Rules

9.1 Input Validation — FluentValidation

Field	Rule	Error Message
Email	Valid format, max 200, unique in DB	Email already registered / Invalid format
Password	Min 8 chars, upper + lower + digit + special	Password does not meet strength requirements
Phone	Indian: optional +91, starts 6-9, 10 digits	Invalid phone number
MenuItem.Price	Greater than 0	Price must be a positive value
PrepTime	Between 1 and 120 minutes	Preparation time must be 1–120 minutes
Order items count	Minimum 1, maximum 20 items per order	Order must contain 1 to 20 items
Item quantity	Between 1 and 10 per line item	Quantity must be between 1 and 10
Status transition	Must follow defined valid path	Invalid status transition from X to Y
Bill generation	Order must be in Ready status	Order must be Ready before generating bill

9.2 Security Rules — Never Trust the Client

- tableId always read from JWT claim for guests — never from request body
- customerId always read from JWT claim for customers — never from request body
- UnitPrice always snapshotted from DB at order time — never accepted from the frontend
- TotalAmount always computed server-side from OrderItems — never accepted from the frontend
- FinalAmt always computed server-side: SubTotal * (1 + TaxRate) - DiscountAmt
- Closed orders (Completed/Cancelled) locked by DB trigger — cannot be modified even if API has a bug
- Rate limit auth endpoints: 10 requests per minute per IP
- Rate limit public endpoints (menu, guest-session): 30 requests per minute per IP
- JWT stored in memory for staff — HttpOnly cookie for refresh token — never localStorage
- Guest token stored in sessionStorage — automatically cleared when tab is closed

10. Inventory Logic

Inventory tracks servings/portions available per dish — not raw ingredients. One Inventory record maps to one MenuItem via MenuItemId. Admin manually sets quantity each day. System decrements on order placement.

```
On order placement:
for each OrderItem:
if inventory.QuantityAvailable < item.Quantity -> reject with message
inventory.QuantityAvailable -= item.Quantity
if QuantityAvailable == 0:
menuItem.IsAvailable = false
inventory.IsAutoDisabled = true
if QuantityAvailable <= LowStockThreshold:
flag for low-stock alert on admin dashboard

On manual restock (PUT /api/inventory/{menuItemId}):
inventory.QuantityAvailable = newQty
if newQty > 0 AND inventory.IsAutoDisabled == true:
menuItem.IsAvailable = true // restore only if stock caused the disable
inventory.IsAutoDisabled = false
// if IsAutoDisabled=false, chef manually disabled it — do NOT auto-restore
```

IsAutoDisabled Flag — Why It Exists

IsAvailable can be false for two reasons: (A) stock hit zero, or (B) Admin/Chef manually toggled it off for a different reason (quality issue, wrong item, etc). The IsAutoDisabled flag tracks whether (A) caused it. When restocking, only restore IsAvailable if IsAutoDisabled is true. If it was a manual disable, restocking should not override the chef's decision.

11. Angular Screens

Screen	Route	Auth	Notes
Login	/auth/login	None	Staff and future customer login
Admin Dashboard	/dashboard	Admin	Live KPIs via SignalR, recent orders, alerts
Menu Browse	/menu	Public	Category tabs, item cards, dietary filter
Menu Admin	/admin/menu	Admin	CRUD with image upload, availability toggle
Category Admin	/admin/categories	Admin	Category list, active toggle, delete
Table Management	/admin/tables	Admin	Floor grid, status color-coded, QR gen
Orders Board	/orders	Admin	Kanban — Pending/InPrep/Ready/Completed
Order Detail	/orders/:id	Admin	Full timeline, item list, notes
Kitchen Display	/kitchen	Chef/Admin	Full-screen dark KDS, timers, overdue alerts
Billing	/billing/:orderId	Admin	Invoice, discount input, payment mode, pay
Inventory	/admin/inventory	Admin/Chef	Stock table, inline edit, low-stock highlight
Reports	/admin/reports	Admin	Revenue line, top items bar, peak hours heatmap
QR Guest Menu	/qr	Guest JWT	Mobile-first, category pills, cart bar, status stepper

11.1 NgRx State Slices

Slice	Manages
AuthState	currentUser, token, sessionType, isAuthenticated
OrderState	active orders, cart items (guest), loading flags
MenuState	categories, items, selected category, search query
KitchenState	queue items, SLA timers, overdue flags

12. Technology Stack

Category	Technology	Version	Purpose
Backend	ASP.NET Core	8.0	REST API + SignalR hub hosting
ORM	Entity Framework Core	8.0	DB access, migrations, query filters
Database	PostgreSQL	16	Primary relational store
Auth	JWT + BCrypt	—	Stateless auth + password hashing
Real-time	SignalR	8.0	Live kitchen and order updates
Validation	FluentValidation	11+	Service layer input validation
Mapping	AutoMapper	12+	Entity to DTO mapping
Frontend	Angular	17+	SPA with lazy-loaded modules
State	NgRx	17+	Reactive state management
UI Components	PrimeNG	Latest	Pre-built accessible components
Charts	ngx-echarts	Latest	Reports dashboard + peak hours heatmap
SignalR Client	@microsoft/signalr	8+	WebSocket client for Angular
Cloud	Microsoft Azure	—	App Service, PostgreSQL, Blob Storage
Phase 2	Docker + Compose	24+	Containerization and local dev stack

13. Implementation Checklist

Phase 1 — Core Scope (Current)

- DB migrations — all 9 entities with Fluent API configs, indexes, query filters
- DB triggers — trg_order_placed, trg_order_completed, trg_generate_bill, trg_lock_closed_orders, trg_guard_item_delete
- DB functions — fn_get_estimated_wait, fn_get_kitchen_queue, fn_daily_revenue
- DB views — vw_active_orders, vw_top_selling_items
- Auth — login, JWT, refresh token, guest-session endpoint, BCrypt, lockout
- Admin register endpoint for creating staff (Chef) accounts
- Role seed data with fixed DateTimeOffset
- Users CRUD
- Tables CRUD + QR code generation (QRCoder library -> Azure Blob)
- Categories CRUD with IsActive toggle
- Menu Items CRUD + IsAvailable toggle + IsDeleted soft delete
- Orders — place order (CanPlaceOrder policy, session branching), status update with transition guard
- Kitchen queue endpoint using fn_get_kitchen_queue() DB function
- Bills — generate (order must be Ready), pay, apply discount
- Inventory — CRUD, auto-toggle IsAvailable on stock change, IsAutoDisabled flag
- SignalR hub — all events wired with correct group targeting
- Background worker — SLA breach detection every 60s, fires OrderOverdue
- FluentValidation validators for all request DTOs
- Global query filters — User.IsDeleted, MenuItem.IsDeleted
- SaveChangesAsync override for auto UpdatedAt
- Angular — all 13 screens including full-screen KDS and mobile QR page
- NgRx store — auth, orders, menu, kitchen state slices
- HTTP interceptor — JWT attachment + 401 auto-refresh
- ngx-echarts reports dashboard — revenue line, top items bar, peak hours heatmap
- Swagger documentation

Phase 2 — Future Scope (Not in Current Build)

- Customer self-registration and login flow
- Customer account-based order placement
- Customer order history screen
- Delivery order type — DeliveryAddress, OutForDelivery status
- Guest session blacklist on order completion (Jti invalidation table)
- Docker + docker-compose for all services
- Azure Service Bus background workers
- Microservices split